Chapter 1

Foundations of Machine Learning: A Summary

Introductory concepts

Machine learning is founded on the principle that data contains hidden patterns reflecting the underlying generative process. The goal is to automatically discover these patterns from observations without explicit programming. This discovery happens through learning from examples, constructing mathematical models that capture relationships in the data, which then serve for prediction, understanding, and decision-making on new, unseen data.

Three Paradigms of Learning

Supervised Learning

- **Definition:** Learning from paired data consisting of input feature vectors and corresponding target values.
- **Goal:** Learn a mapping from observations to targets to predict labels for new inputs.
- **Formalization:** Define a model as either:
  - A direct function $y(\mathbf{x})$ that outputs a prediction.
  - A probability distribution $p(t|\mathbf{x})$ quantifying uncertainty about the target.
- **Types:**

  Regression: Target is a continuous real number (e.g., prices, temperatures).

  Classification: Target is a discrete category (e.g., spam detection, digit recognition), further divided into binary ($|\mathcal{Y}| = 2$) and multi-class ($|\mathcal{Y}| > 2$).

Unsupervised Learning

- **Definition:** Learning from data without explicit target values.
- **Goal:** Uncover inherent structure, understand data nature, and organize it meaningfully.
- **Main Forms:**
  - Clustering: Grouping similar items to reveal natural categories.
  - Density estimation: Modeling the underlying probability distribution that generated the data.
  - Dimensionality reduction: Projecting high-dimensional data onto lower-dimensional representations while preserving essential information.
  - Generative modeling: Learning to generate new synthetic data points that appear to come from the same underlying process as training data.

- **Applications:** Identify outliers and anomalies, facilitate visualization, provide foundation for feature engineering.

Reinforcement Learning

- **Definition:** Learning through dynamic interaction with an environment.
- **Process:** An agent observes the current **state**, selects an **action**, receives a **reward**, and observes a new state.
- **Goal:** Discover an optimal **policy** $\pi : S \to A$ that maximizes expected cumulative reward.
- **Optimization Objective:**

$$\pi^* = \underset{\pi}{\operatorname{argmax}}\, \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \tag{1.1}$$

  where $\gamma \in [0, 1]$ is a discount factor.
- **Key Challenge:** Exploration-exploitation tradeoff.
- **Note:** This paradigm employs distinct mathematical techniques and will be set aside in favor of supervised and unsupervised learning.

Data Representation

- **Feature Vector:** A collection $\mathbf{x} = (x_1, x_2, \dots, x_d)$ of d numerical measurements describing an object.
- **Importance:** Choice of features affects both prediction quality and computational efficiency.
- **Feature Engineering:** The process of defining new features through functional combinations, selecting the most informative subset, or learning better representations. This is often as important as the choice of the learning algorithm itself.

The Learning Process

- **Training Set:** $\mathcal{T} = (\mathbf{X}, \mathbf{t})$ containing n examples.
  - Supervised: $\mathcal{T} = \{(\mathbf{x}_1, t_1), \dots, (\mathbf{x}_n, t_n)\}$.
  - Unsupervised: $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$.
- **Organization:** Feature matrix $\mathbf{X}$ (objects as rows) and target vector $\mathbf{t}$ (corresponding labels).
- **Core Challenge:** **Generalization** — performing well on new, unseen data, not just on the training set.
- **Key Concepts:**
  - **Capacity/Complexity**: Model flexibility; more complex models fit intricate patterns but risk overfitting.
  - **Bias-variance tradeoff**: Tension between underfitting (models too simple) and overfitting (models adapting too closely to training noise).
  - **Regularization**: Techniques that penalize model complexity to encourage simpler, better-generalizing solutions.

Formal Definition of a Supervised Machine Learning Task

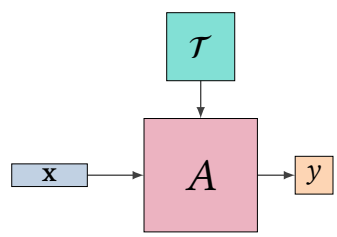
Core Components

- **Domain Set $\mathcal{X}$:** Collection of objects to label or make predictions about. Each object $\mathbf{x} \in \mathcal{X}$ is typically modeled as $\mathbf{x} = (x_1, x_2, \dots, x_d)$ where d is the **dimensionality**.
- **Label Set $\mathcal{Y}$:** All possible output values.
  - $\mathcal{Y}$ continuous $\rightarrow$ **regression**.
  - $\mathcal{Y}$ discrete $\rightarrow$ **classification**:
    * Binary: $|\mathcal{Y}| = 2$.
    * Multi-class: $|\mathcal{Y}| > 2$.
- **Prediction Rule:** A function $h : \mathcal{X} \mapsto \mathcal{Y}$ (classifier or regressor) returned by the learning algorithm $\mathcal{A}$.
- **Prediction Strategies:**
  1. Direct target value prediction: $y = h(\mathbf{x})$.
  2. Probability distribution prediction: Output $p(y|\mathbf{x})$; then apply decision rule (e.g., select label with highest probability).

Three Approaches to Deriving a Predictor

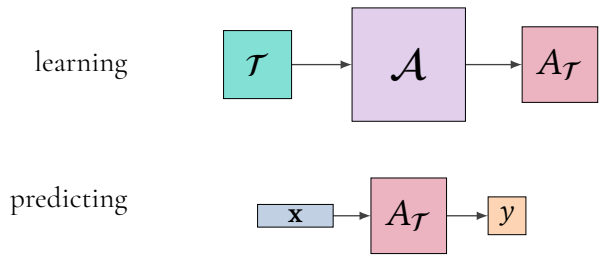
Direct Prediction Computation

- **Mechanism:** Apply algorithm A at prediction time using the entire training set: $y = h(\mathbf{x}, \mathcal{T})$.
- **Characteristics:** No explicit learning phase; training examples are stored and referenced during prediction.
- **Example:** k-nearest neighbors — find k closest objects in $\mathbf{X}$ to $\mathbf{x}$, predict majority class. No explicit model of the data.
- **Visualization:**



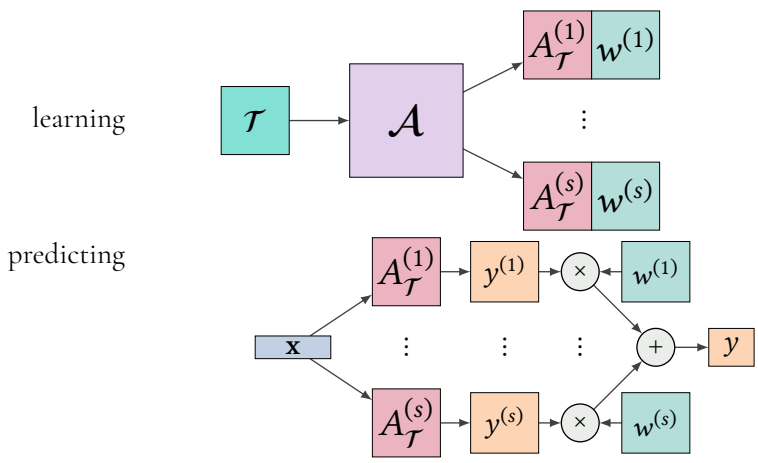
Model-Based Learning Approach

- **Mechanism:** Define a **hypothesis class** $\mathcal{H}$ (family of functions $\mathcal{X} \rightarrow \mathcal{Y}$). Learning algorithm $\mathcal{A}$ selects $h_{\mathcal{T}} \in \mathcal{H}$ that best approximates training examples.
- **Prediction:** Apply $h_{\mathcal{T}}(\mathbf{x})$.
- **Parametric Functions:** $\mathcal{H}_{\boldsymbol{\theta}}$ where $\boldsymbol{\theta} = (\theta_1, \dots, \theta_m)$. Learning finds optimal parameter values.
- **Example:** Linear regression: $y = \sum_{i=1}^{d} w_i x_i + w_0 = \mathbf{w}^T \overline{\mathbf{x}}$ where $\mathbf{w} = (w_0, w_1, \dots, w_d)^T$ and $\overline{\mathbf{x}} = (1, x_1, \dots, x_d)^T$.
- **Visualization:**



Ensemble Learning Approach

- **Mechanism:** Construct multiple predictors and combine their predictions.
- **Process:**
  1. Derive s different predictors $A_{\mathcal{T}}^{(1)}, \dots, A_{\mathcal{T}}^{(s)}$ from training set.
  2. Derive weight $w^{(i)}$ for each predictor reflecting its quality.
  3. For new item $\mathbf{x}$, gather predictions $y^{(i)} = h_{\mathcal{T}}^{(i)}(\mathbf{x})$ and combine using weights.
- **Combination Methods:**
  - Regression (weighted average): $h(\mathbf{x}) = \sum_{i=1}^{s} w^{(i)} h^{(i)}(\mathbf{x})$ where $\sum_{i=1}^{s} w^{(i)} = 1$, $w^{(i)} \geq 0$.
  - Classification (weighted voting): $h(\mathbf{x}) = \underset{y \in \mathcal{Y}}{\operatorname{argmax}} \sum_{i=1}^{s} w^{(i)} \mathbb{1}[h^{(i)}(\mathbf{x}) = y]$.
- **Fully Bayesian Prediction:** Average predictions over all possible parameter values (integral over parameter space).
- **Visualization:**



Probabilistic Framework for Machine Learning

Data Generation Assumption

- Objects are sampled independently from unknown probability distribution p_M over $\mathcal{X}$.
- Labels arise from conditional probability distribution $p_C(t|\mathbf{x})$ (captures noise, ambiguity).
- Joint distribution: $p(\mathbf{x}, t) = p_M(\mathbf{x}) p_C(t|\mathbf{x})$.

- **Simplification:** Often assume unknown ground truth function $f : \mathcal{X} \to \mathcal{Y}$ with $t = f(\mathbf{x})$ (deterministic case).

Risk and Loss

Loss Function

- **Definition:** $L : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$ quantifies penalty for predicting y when true label is t.
- **Example (0-1 loss for classification):**

$$L(y, t) = \mathbb{1}[y \neq t] \tag{1.2}$$

Prediction Risk (for a single object x)

- Deterministic setting (true function f):

$$\mathcal{R}_f(y, \mathbf{x}) \triangleq L(y, f(\mathbf{x})) \tag{1.3}$$

- Probabilistic setting (conditional distribution p_C):

$$\mathcal{R}_{p_C}(y, \mathbf{x}) \triangleq \mathop{\mathbb{E}}_{t \sim p_C(\cdot|\mathbf{x})} [L(y, t)] = \int_{\mathcal{Y}} L(y, t)\, p_C(t|\mathbf{x})\, dt \tag{1.4}$$

  For discrete $\mathcal{Y}$:

$$\mathcal{R}_{p_C}(y, \mathbf{x}) \triangleq \sum_{t \in \mathcal{Y}} L(y, t)\, p_C(t|\mathbf{x}) \tag{1.5}$$

Expected Risk (True Risk)

- Average loss over infinitely many examples from true distribution:
  - Deterministic:

$$\mathcal{R}_{p_M, f}(h) \triangleq \mathop{\mathbb{E}}_{\mathbf{x} \sim p_M} [L(h(\mathbf{x}), f(\mathbf{x}))] = \int_{\mathcal{X}} L(h(\mathbf{x}), f(\mathbf{x}))\, p_M(\mathbf{x})\, d\mathbf{x} \tag{1.6}$$

  - Probabilistic:

$$\mathcal{R}_p(h) \triangleq \mathop{\mathbb{E}}_{(\mathbf{x}, t) \sim p} [L(h(\mathbf{x}), t)] = \int_{\mathcal{X}} \int_{\mathcal{Y}} L(h(\mathbf{x}), t)\, p_M(\mathbf{x})\, p_C(t|\mathbf{x})\, d\mathbf{x}\, dt \tag{1.7}$$

- With 0-1 loss:

$$\mathcal{R}_{p_M, f}(h) = \mathop{\mathbb{P}}_{\mathbf{x} \sim p_M} [h(\mathbf{x}) \neq f(\mathbf{x})] \tag{1.8}$$

- **Problem:** True distributions p_M and p_C (or f) are unknown; expected risk cannot be directly computed.

Empirical Risk

- Estimate based on training set $\mathcal{T}$:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) \triangleq \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} L(h(\mathbf{x}), t) \tag{1.9}$$

- With 0-1 loss (error rate):

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \mathbb{1}[h(\mathbf{x}) \neq t] = \frac{|\{(\mathbf{x},t) \in \mathcal{T} \mid h(\mathbf{x}) \neq t\}|}{|\mathcal{T}|} \tag{1.10}$$

The Learning Problem as Optimization

- **Strategy:** Minimize empirical risk instead of unknown expected risk.
- **Formalization:** Given hypothesis class $\mathcal{H}$ (inductive bias):

$$h_{\mathcal{T}} = \operatorname*{argmin}_{h\in\mathcal{H}} \overline{\mathcal{R}}_{\mathcal{T}}(h) \tag{1.11}$$

- **Caveat:** Minimizing empirical risk does not guarantee good performance on unseen data (overfitting).
- **Interpretation:** The probabilistic framework transforms "learning from data" into finding a function in a specified class that best explains observed data according to a chosen loss function, while acknowledging this is only a proxy for minimizing expected risk on future data.

Inductive Bias

Definition and Importance

- **Inductive Bias:** The set of assumptions embodied in the choice of hypothesis class $\mathcal{H}$ about what kind of predictor is likely to work well.
- **Core Tradeoff:**
  - If $\mathcal{H}$ is too small: Risk excluding the true underlying function (underfitting).
  - If $\mathcal{H}$ is too large: Risk finding a function that fits training data perfectly but performs poorly on new data (overfitting).

Overfitting

- **Definition:** Phenomenon where predictor fits training set perfectly but performs poorly on new examples.
- **Example:** Binary classification with $p_C(t = 1|\mathbf{x}) = \frac{1}{2}$ for all objects. Choose $\mathcal{H}$ as all possible functions $\mathcal{X} \to \{0, 1\}$. Define memorizer:

$$h_{\mathcal{T}}(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{x} = \mathbf{x}_i \in \mathbf{X} \text{ and } t_i = 1 \\ 0 & \text{otherwise} \end{cases} \tag{1.12}$$

- **Analysis:**
  - Empirical risk on $\mathcal{T} = 0$ (perfect training accuracy).
  - For new objects (not in training set), $h_{\mathcal{T}}$ outputs 0, predicting minority class.
  - Since classes equally likely, expected risk $\approx \frac{1}{2}$ (random guessing).
- **Cause:** $\mathcal{H}$ is so large that algorithm can find functions achieving perfect empirical risk despite high expected risk.

Underfitting

- **Definition:** Phenomenon where hypothesis class is too restrictive to capture the true relationship.
- **Example:** True function is highly nonlinear, but $\mathcal{H}$ restricted to linear predictors.
- **Consequence:** Both training and test errors remain unacceptably high; no gap between them because predictor is too rigid to memorize.

The Bias-Variance Tradeoff

Setup

- Consider generating different training sets $\mathcal{T}_1, \mathcal{T}_2, \ldots$ from true distribution p_M and training algorithm $\mathcal{A}$ on each.
- Each yields different predictors $h_{\mathcal{T}_1}, h_{\mathcal{T}_2}, \ldots$.

Bias

- **Definition:** Measures how far the average predictor (averaged over many training sets) is from the true underlying function.
- **Interpretation:** Systematic error; predictors tend to be systematically wrong in the same direction.
- **Cause:** Restrictive hypothesis classes that cannot represent complex patterns.
- **Example:** Linear model applied to highly nonlinear problem has high bias regardless of data amount.

Variance

- **Definition:** Measures how much the learned predictor fluctuates depending on which particular training set was received.
- **Interpretation:** Sensitivity to specific noise and quirks of training data; signature of overfitting.
- **Cause:** Large, flexible hypothesis classes where small variations in training set lead to dramatically different learned functions.

The Tradeoff

- **Small, restrictive $\mathcal{H}$:** Reduces variance (class not flexible enough to chase noise) but increases bias (cannot represent true function).

- **Large, expressive $\mathcal{H}$:** Reduces bias (can approximate almost any function) but increases variance (can fit training noise).
- **Optimal Hypothesis Class:** Balances these concerns; size depends on:
  - Complexity of true underlying function.
  - Amount of training data available.
  - Noise in observations.
- **Manifestation:**
  - Small datasets: Flexible models may perform well if they capture true pattern before noise.
  - Large datasets: Flexible models begin to overfit (memorize different noise patterns).
  - Restrictive models: May never fit training data well regardless of data amount.

Computational and Statistical Considerations

- Even if true function lies in $\mathcal{H}$, finding the predictor that minimizes empirical risk may be computationally intractable.
- Some hypothesis classes lead to NP-hard optimization problems.
- **Practical Consequence:** May deliberately choose smaller/different $\mathcal{H}'$ to make learning solvable in reasonable time.
- **Implication:** Inductive bias reflects both statistical beliefs about the problem and computational pragmatism.